Práctica 8: Programación con Python

Sistema «tienda» (punto de venta sin base de datos)

Licenciatura en Ingeniería de Software · Escuela Superior de Tlahuelilpan, UAEH

Alumno: Yamelin Pérez Barrera

1. Introducción

Python es un lenguaje de programación de propósito general creado a principios de los años noventa por Guido van Rossum. Es software libre, multiplataforma, interpretado, de tipado dinámico y fuertemente tipado, y soporta varios paradigmas: orientado a objetos, estructurado, imperativo y, en menor medida, funcional. Su sintaxis hace que la indentación sea parte de la estructura del código, lo que favorece programas legibles.

La biblioteca estándar de Python incluye **Tkinter**, un módulo para construir interfaces gráficas de escritorio (GUI) sin instalar paquetes adicionales. En esta práctica se usó Tkinter, junto con su extensión `ttk`, para desarrollar un sistema de punto de venta llamado **tienda**, que permite administrar clientes y artículos, registrar ventas y generar un ticket digital con subtotal, IVA y total.

Un punto de venta es una aplicación típica para practicar los fundamentos del lenguaje: estructuras de datos (listas, diccionarios, tuplas), estructuras de control, funciones, clases, manejo de errores y diseño de interfaces. Conforme a las indicaciones, el sistema no utiliza base de datos: toda la información se genera y se conserva en memoria mientras el programa está en ejecución.

2. Objetivo

Objetivo general. Desarrollar una interfaz visual en Python que permita hacer uso de diversos elementos de este paradigma de programación.

Objetivos específicos.

- Programar un sistema similar a un punto de venta sin bases de datos.
- Generar las interfaces necesarias para manipular la información de entrada, dentro del sistema y de salida.
- Crear interfaces claras, intuitivas y atractivas para los usuarios.

3. Desarrollo de la actividad práctica

3.1 Requerimientos

- Almacenar información de clientes, de artículos y de las ventas realizadas.
- Cada venta genera un ticket con: producto(s), cantidad, precio unitario e importe (cantidad × precio).
- Al final del ticket: Subtotal = suma de importes; IVA = Subtotal × 0.16; Total = Subtotal + IVA.
- Inicio de sesión con identificación de roles.
- Sin base de datos: los datos se generan desde el programa.

3.2 Herramientas

Equipo de cómputo con Python 3.12 y el módulo estándar `tkinter` (`ttk`). El programa está contenido en un solo archivo, `tienda.py`, y se ejecuta con `python tienda.py`.

3.3 Diseño del sistema

El programa se organizó en tres capas dentro del mismo archivo:

- **Datos en memoria:** los diccionarios `USUARIOS`, `CLIENTES`, `PRODUCTOS` y la lista `VENTAS`, cargados con información de ejemplo.
- **Lógica:** funciones independientes de la interfaz (`siguiente_id`, `calcular_totales`, `generar_ticket`), lo que facilita probarlas por separado.
- **Interfaz:** clases que heredan de widgets de Tkinter (`Login`, `App`, `CrudFrame`, `VentaFrame`, `HistorialFrame`).

3.4 Roles y control de acceso

La ventana `Login` valida usuario y contraseña contra el diccionario `USUARIOS`. Según el rol, la ventana principal `App` construye pestañas distintas:

Usuario	Contraseña	Rol	Módulos disponibles
admin	admin123	Administrador	Clientes (alta, edición y baja), Artículos, Ventas, Historial de ventas
vendedor	venta123	Vendedor	Clientes (alta y edición), Ventas

3.5 Estructuras de datos

Estructura	Contenido
`CLIENTES` (dict)	Clave: ID del cliente. Valor: diccionario con nombre, teléfono, correo y dirección.
`PRODUCTOS` (dict)	Clave: código del producto (desde 101). Valor: nombre, categoría, precio y existencia.
`VENTAS` (list)	Una venta por elemento: folio, fecha, cliente, vendedor, detalle, subtotal, IVA y total.
`carrito` (list)	Renglones de la venta en curso: código, nombre, cantidad, precio e importe.

3.6 Descripción de los módulos

Inicio de sesión (`Login`). Muestra los campos Usuario y Contraseña (esta última oculta con asteriscos). Si las credenciales son incorrectas muestra un mensaje de error; si son correctas, abre la ventana principal.

Clientes y Artículos (`CrudFrame`). Una sola clase genérica construye ambas pantallas a partir de una lista de campos (`clave, etiqueta, tipo`). Muestra una tabla (`ttk.Treeview`) y un formulario con los

botones Agregar, Editar, Eliminar y Limpiar. Valida que ningún campo esté vacío, que los campos numéricos sean válidos y que no sean negativos. El ID se asigna automáticamente.

Ventas (`VentaFrame`). El usuario elige un cliente, selecciona un código de producto de la lista (tomada de la tabla de productos), captura la cantidad y agrega el renglón al ticket. El sistema valida que la cantidad sea un entero positivo y que alcance la existencia; si el producto ya está en el ticket, suma las cantidades. Los totales se actualizan en pantalla en cada cambio. Al finalizar, se descuenta la existencia, se guarda la venta y se muestra el ticket.

Historial (`HistorialFrame`, solo administrador). Lista todas las ventas con folio, fecha, cliente, vendedor y total, muestra el número de ventas e ingresos acumulados y permite reabrir cualquier ticket con doble clic.

3.7 Código relevante

Cálculo de totales, según las fórmulas de la práctica:

```
IVA = 0.16

def calcular_totales(detalle):
    subtotal = round(sum(d['importe'] for d in detalle), 2)
    iva = round(subtotal * IVA, 2)
    total = round(subtotal + iva, 2)
    return subtotal, iva, total
```

Validación de existencia al agregar un producto al ticket:

```
ya = next((d for d in self.carrito if d['codigo'] == codigo), None)
if cant + (ya['cantidad'] if ya else 0) > prod['existencia']:
    return messagebox.showwarning('Sin existencia', 'No hay suficientes piezas.')
```

Ciclo principal que permite cerrar sesión y volver al login:

```
def main():
    while True:
        login = Login()
        login.mainloop()
        if not login.resultado:
            break
        app = App(*login.resultado)
        app.mainloop()
        if not app.cerrar_sesion:
            break
```

3.8 Pruebas realizadas

La interfaz usa una paleta roja (botones, encabezados de tabla y pestañas activas en rojo sobre un fondo rosa claro), aplicada con estilos de ttk en el tema «clam»; el login y el ticket usan los mismos colores. Se ejecutó el programa y se realizó una venta de prueba con el usuario administrador. También se comprobó que el rol Vendedor solo ve las pestañas Clientes y Ventas. El archivo `tienda.py` se entrega junto con este reporte.

4. Resultados

4.1 Venta de prueba

Cliente: Ana López (ID 1). Productos agregados al ticket:

Código	Producto	Cantidad	Precio unitario	Importe
101	Cuaderno profesional	2	$45.50	$91.00
103	Mochila escolar	1	$350.00	$350.00
104	Memoria USB 32GB	3	$120.00	$360.00

Subtotal = 91.00 + 350.00 + 360.00 = **$801.00**; IVA = 801.00 × 0.16 = **$128.16**; Total = 801.00 + 128.16 = **$929.16**.

4.2 Ticket generado por el sistema

```
==================================================
                      TIENDA
                  Nota de venta
==================================================
Folio   : 00001
Cliente : Ana López
Atendió : admin
--------------------------------------------------
Producto               Cant     P.Unit     Importe
--------------------------------------------------
Cuaderno profesional      2      45.50       91.00
Mochila escolar           1     350.00      350.00
Memoria USB 32GB          3     120.00      360.00
--------------------------------------------------
                             Subtotal:      801.00
                            IVA (16%):      128.16
                                TOTAL:      929.16
==================================================
             ¡Gracias por su compra!
```

4.3 Efecto en el inventario

Producto	Existencia inicial	Vendidas	Existencia final
Cuaderno profesional (101)	40	2	38
Mochila escolar (103)	15	1	14
Memoria USB 32GB (104)	25	3	22

4.4 Resumen de funcionalidades verificadas

Funcionalidad	Resultado
Inicio de sesión con credenciales correctas e incorrectas	Correcto
Roles: el Vendedor solo accede a Clientes y Ventas	Correcto
Alta, edición y baja de clientes y artículos con validación	Correcto
Selección de código de producto desde la tabla de productos	Correcto
Cálculo de importe, subtotal, IVA (16 %) y total	Correcto
Descuento de existencias y bloqueo si no hay stock	Correcto
Historial de ventas y reapertura de tickets	Correcto

5. Discusión

El sistema cumple los requerimientos de la práctica: administra clientes, artículos y ventas, genera un ticket digital con detalle, subtotal, IVA y total, e incluye inicio de sesión por roles, todo sin base de datos. El resultado de la venta de prueba coincide con el cálculo manual (801.00 + 128.16 = 929.16).

Separar la lógica de la interfaz fue una decisión útil: `calcular_totales` y `generar_ticket` se pueden probar sin abrir ninguna ventana. Reutilizar `CrudFrame` para clientes y artículos evitó duplicar código y mostró el valor de la orientación a objetos y de los parámetros configurables.

Usar `Decimal` para los importes sería lo ideal en un sistema real, porque los números `float` pueden acumular errores de representación. Aquí se usó `round(..., 2)` para mantener la práctica sencilla.

Las limitaciones más importantes son la **persistencia** y la **seguridad**. Al cerrar el programa se pierden las ventas y los cambios, porque los datos viven en memoria, y las contraseñas están escritas en el código en texto plano. Estas limitaciones son aceptables para la práctica, pero en un sistema real se resolverían con una base de datos y con contraseñas almacenadas como hash. Como mejoras futuras se pueden agregar exportación del ticket a PDF o texto, administración de usuarios, búsqueda y filtros en las tablas, y devoluciones.

6. Cuestionario

1. ¿Qué paradigmas de programación utiliza Python?

Python es multiparadigma. Soporta la **programación orientada a objetos** (clases, herencia y polimorfismo; en esta práctica `App`, `Login` y `CrudFrame` heredan de widgets de Tkinter), la **programación estructurada** (secuencia, selección e iteración), la **programación imperativa** (instrucciones que cambian el estado del programa) y, en menor medida, la **programación funcional** (funciones de primer orden, `lambda`, `map`, `filter`, comprensiones).

2. ¿Cuál es la clasificación de los tipos de datos estándar en Python?

Categoría	Tipos
Numéricos	`int`, `float`, `complex`
Booleano	`bool` (`True`, `False`)
Secuencias	`str`, `list`, `tuple`, `range` (y `bytes`, `bytearray`)
Mapeos	`dict`
Conjuntos	`set`, `frozenset`
Nulo	`NoneType` (`None`)

Se pueden clasificar también en **mutables** (`list`, `dict`, `set`) e **inmutables** (`int`, `float`, `str`, `tuple`, `frozenset`, `bool`).

3. ¿Cuáles son las palabras reservadas en Python?

En Python 3.12 existen 35 palabras reservadas:

Palabra	Descripción
`False` / `True`	Valores booleanos.
`None`	Representa la ausencia de valor.
`and` / `or` / `not`	Operadores lógicos.
`as`	Asigna un alias (`import ... as`, `with ... as`, `except ... as`).
`assert`	Comprueba una condición; lanza `AssertionError` si es falsa.
`async` / `await`	Definen y esperan corrutinas (programación asíncrona).
`break`	Sale de un ciclo.
`class`	Define una clase.

Palabra	Descripción
`continue`	Salta a la siguiente iteración del ciclo.
`def`	Define una función.
`del`	Elimina una variable, elemento o fragmento.
`if` / `elif` / `else`	Estructura condicional. `else` también se usa en ciclos y en `try`.
`for` / `while`	Ciclos de iteración.
`in`	Pertenencia a una colección; también recorre elementos en `for`.
`is`	Comprueba identidad de objetos.
`try` / `except` / `finally`	Manejo de excepciones. `finally` siempre se ejecuta.
`raise`	Lanza una excepción.
`from`	Importa nombres específicos de un módulo; se usa también en `yield from` y `raise ... from`.
`import`	Importa módulos.
`global`	Declara que una variable pertenece al ámbito global.
`nonlocal`	Declara una variable del ámbito de la función contenedora.
`lambda`	Crea funciones anónimas.
`pass`	Instrucción vacía, no hace nada.
`return`	Devuelve un valor desde una función.
`with`	Administra contextos (p. ej. apertura y cierre de archivos).
`yield`	Convierte una función en generador devolviendo valores uno a uno.

4. ¿Qué operadores de asignación se pueden utilizar en Python?

El operador básico es `=`. Los compuestos combinan una operación con la asignación: `+=`, `-=`, `*=`, `/=`, `//=` (división entera), `%=` (módulo), `**=` (potencia), `&=`, `|=`, `^=`, `>>=` y `<<=` (operaciones a nivel de bit). Además existe el operador morsa `:=` (Python 3.8+), que asigna un valor dentro de una expresión. Por ejemplo, `ya['cantidad'] += cant` o `existencia -= cantidad`.

5. ¿Cómo se realizan los comentarios de línea y multilínea en Python?

El comentario de línea comienza con `#`: todo lo que sigue hasta el fin de la línea se ignora. Python no tiene sintaxis específica de comentario multilínea; se escribe un `#` al inicio de cada línea o se usa una cadena de triple comilla (`'''...'''` o `"""..."""`) que, al no asignarse a una variable, funciona como comentario. Si se coloca justo al inicio de un módulo, clase o función es un *docstring* (documentación); `tienda.py` comienza con uno.

6. ¿Qué es una lista en Python y qué métodos utiliza?

Una lista es una secuencia **ordenada y mutable** que puede contener elementos de cualquier tipo, incluso repetidos. Se define con corchetes: `[1, 'a', 3.5]`. En la práctica se usó para `VENTAS` y para el carrito de compra.

Método	Descripción
`append(x)`	Agrega un elemento al final.
`extend(iterable)`	Agrega todos los elementos de otro iterable.
`insert(i, x)`	Inserta `x` en la posición `i`.
`remove(x)`	Elimina la primera aparición de `x`.
`pop([i])`	Elimina y devuelve el elemento en `i` (por defecto el último).
`clear()`	Vacía la lista (se usó en `carrito.clear()`).
`index(x)`	Devuelve la posición de la primera aparición de `x`.
`count(x)`	Cuenta cuántas veces aparece `x`.
`sort()`	Ordena la lista en su lugar.
`reverse()`	Invierte el orden en su lugar.
`copy()`	Devuelve una copia superficial.

7. ¿Qué es una tupla y cuáles son sus métodos?

Una tupla es una secuencia **ordenada e inmutable**: sus elementos no pueden modificarse, agregarse ni eliminarse después de crearla. Se define con paréntesis: `(1, 2, 3)`. Se usó, por ejemplo, en la definición de columnas `('codigo', 'nombre', ...)` y en los campos `(clave, etiqueta, tipo)` de `CrudFrame`. Como es inmutable, solo tiene dos métodos:

- `count(x)`: devuelve cuántas veces aparece `x`.
- `index(x)`: devuelve el índice de la primera aparición de `x`.

8. ¿Qué son los conjuntos en Python y cuáles son sus métodos?

Un conjunto (`set`) es una colección **no ordenada, mutable y sin elementos repetidos**. Se define con llaves (`{1, 2, 3}`) o con `set()`; el conjunto vacío se crea con `set()`, porque `{}` crea un diccionario. Sus métodos son:

Método	Descripción
`add(x)`	Agrega un elemento.
`update(otro)`	Agrega los elementos de otro iterable.
`remove(x)`	Elimina `x`; lanza `KeyError` si no existe.
`discard(x)`	Elimina `x` si existe, sin error.
`pop()`	Elimina y devuelve un elemento arbitrario.
`clear()`	Vacía el conjunto.
`copy()`	Devuelve una copia.
`union(otro)`	Unión de conjuntos.
`intersection(otro)`	Elementos comunes.
`difference(otro)`	Elementos que están en uno y no en el otro.
`symmetric_difference(otro)`	Elementos que están en uno u otro, pero no en ambos.
`intersection_update`, `difference_update`, `symmetric_difference_update`	Versiones que modifican el conjunto original.
`issubset(otro)` / `issuperset(otro)`	Indican si es subconjunto o superconjunto.
`isdisjoint(otro)`	Indica si no hay elementos en común.

9. ¿Qué estructuras de control utilizaste en el desarrollo de la práctica?

- **Selectiva:** `if` / `elif` / `else` en la validación de credenciales, en la comprobación de existencias, en la creación de pestañas según el rol (`if admin:`) y en la validación de formularios.
- **Repetitiva `for`:** para recorrer los renglones del ticket en `generar_ticket`, llenar las tablas `Treeview` y construir los campos del formulario con `enumerate`.
- **Repetitiva `while`:** `while True` en `main()` para volver al inicio de sesión después de cerrar sesión.
- **Manejo de excepciones:** `try` / `except ValueError` al convertir texto a `int` o `float`, y `raise ValueError` para señalar datos inválidos.
- **Control de flujo:** `return` para salir anticipadamente de las funciones tras mostrar un mensaje de error.
- También se emplearon comprensiones de listas y expresiones generadoras (`sum(d['importe'] for d in detalle)`).

10. Si utilizaste alguna de las funciones de Python, describe el procedimiento realizado para utilizarla.

Se definieron funciones propias con `def`: el procedimiento fue declarar el nombre y los parámetros, escribir el cuerpo con sangría, usar `return` para devolver el resultado y después llamarla desde la interfaz.

- **`calcular_totales(detalle)`:** recibe la lista de renglones del ticket; suma los importes con `sum()`, calcula el IVA multiplicando por 0.16 y devuelve la tupla `(subtotal, iva, total)`. La interfaz la llama cada vez que cambia el carrito (`s, i, t = calcular_totales(self.carrito)`) y al finalizar la venta.
- **`generar_ticket(venta)`:** recibe el diccionario de la venta, arma una lista de líneas de texto formateadas con f-strings (por ejemplo `{valor:>12.2f}` para alinear a la derecha con dos decimales) y las une con `'\n'.join(lineas)`. El texto resultante se inserta en un widget `Text`.
- **`siguiente_id(datos)`:** usa `max(datos.keys(), default=...) + 1` para asignar el siguiente ID o folio.

También se usaron funciones integradas de Python: `sum()`, `round()`, `max()`, `sorted()`, `next()`, `int()`, `float()`, `len()` y `enumerate()`. Por ejemplo, `round(subtotal * IVA, 2)` redondea el IVA a dos decimales, y `sorted(CLIENTES.items())` ordena los registros por ID antes de mostrarlos en las tablas.

7. Referencias bibliográficas

- Covatec (2018). *Programación en Python, nivel básico*. Disponible en: https://entrenamiento-python-basico.readthedocs.io/es/latest/index.html
- López Ostenero, F. y García Serrano, A. M. (2014). *Teoría de los Lenguajes de Programación*. Editorial Universitaria Ramón Areces. ISBN: 9788499611396.
- Python Software Foundation. *The Python Standard Library: tkinter, Python interface to Tcl/Tk*. https://docs.python.org/3/library/tkinter.html